CSCE 548 – Project 4

Security Vulnerability Database

Ky Nguyen

---

1. Introduction

This project implements a **Security Vulnerability Database** using an **n-tier architecture**. The system allows users to retrieve, insert, update, and remove vulnerability-related records through a web client interface connected to a backend REST API.

The backend service communicates with a **MySQL database** through a layered architecture consisting of a service layer, business logic layer, and data access layer. The client interface sends HTTP requests to the API, which processes the requests and interacts with the database.

The purpose of this project is to demonstrate the implementation and deployment of a **full-stack application using generative AI assistance**, building upon the architecture developed in Projects 1–3.

---

2. System Architecture

The application follows an **n-tier architecture** consisting of four layers:

Client Layer → Service Layer → Business Layer → Data Layer → Database

Client Layer

Provides a web-based user interface that allows users to interact with the system.

Service Layer

A Flask-based REST API that processes HTTP requests and routes them to the appropriate business logic.

Business Layer

Handles application logic and validation before interacting with the database.

Data Layer

Implements Data Access Objects (DAO) responsible for executing SQL queries.

Database

A MySQL database that stores vulnerability records and related information.

---

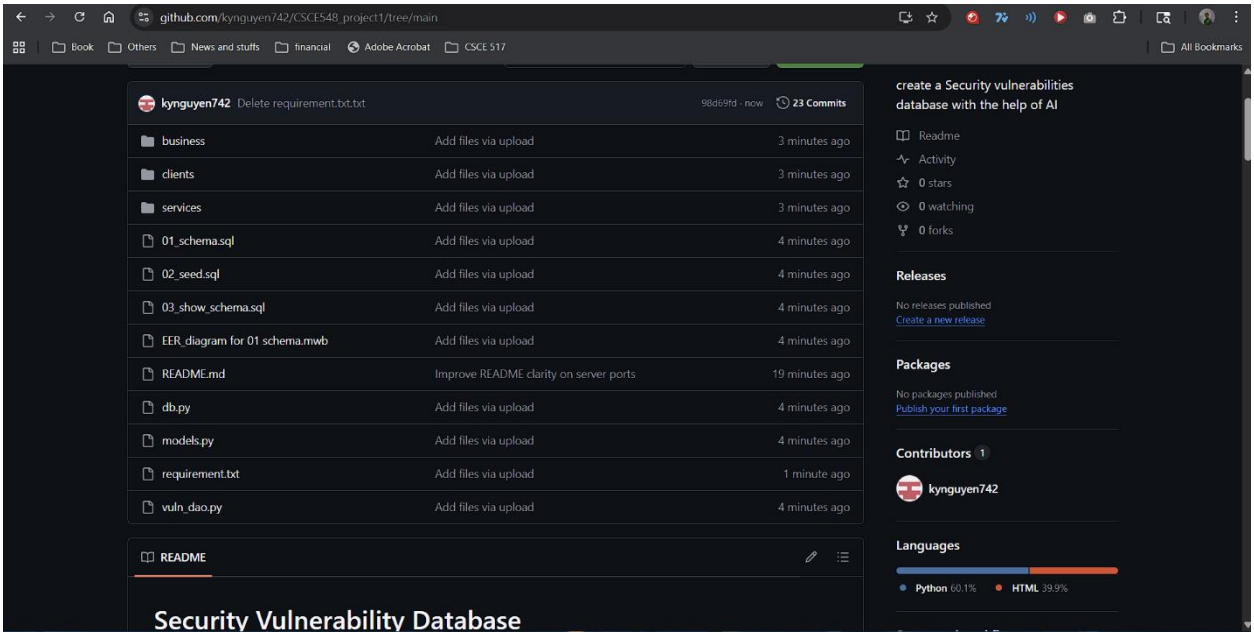
3. Deployment Instructions

This section describes how to download, configure, and run the project from the GitHub repository.

3.1 Download the Repository

The project source code is available on GitHub.

Figure 1 — GitHub repository containing the Security Vulnerability Database project

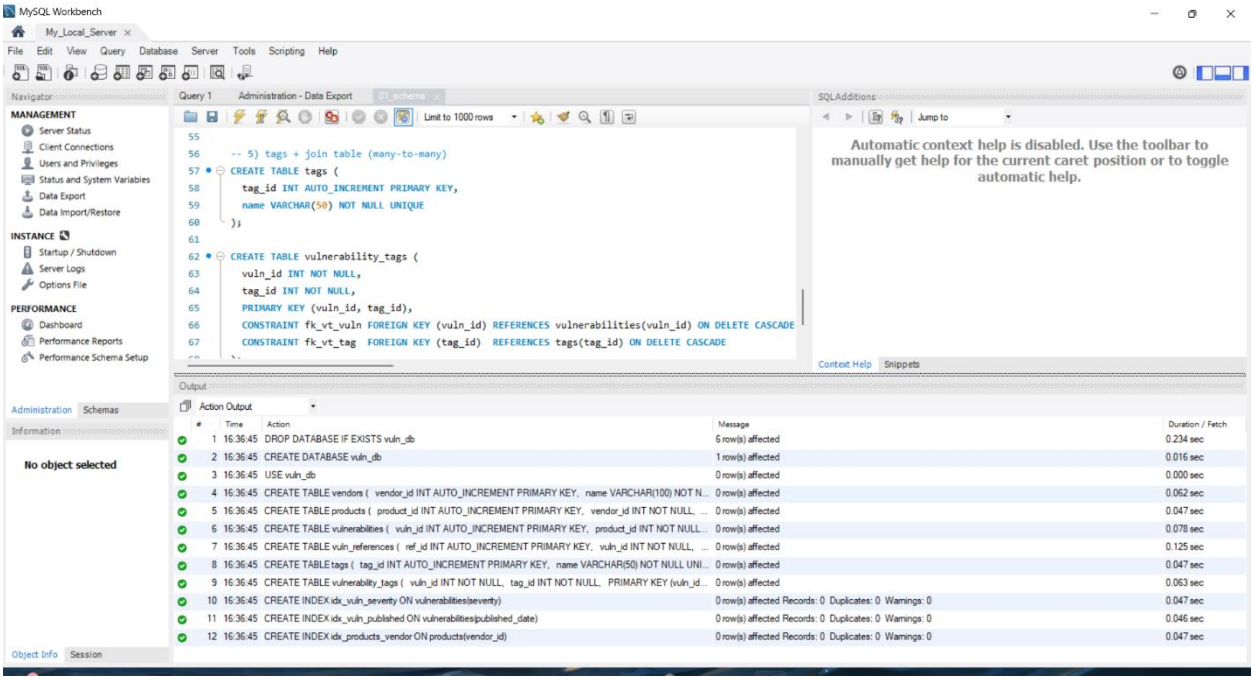


The project source code is stored in a GitHub repository. The repository contains the implementation of the client, service, business, and data layers, as well as SQL scripts used to create and populate the database. The repository structure includes folders for the business logic, service API, and client interface. The root directory also contains database schema scripts, configuration files, and documentation required to run the system.

3.2 Database Setup

The database schema was created using the SQL script 01_schema.sql executed in MySQL Workbench. The script creates the main database (vuln_db) and defines all required tables and relationships used by the application. These include tables for vendors, products, vulnerabilities, vulnerability references, tags, and the many-to-many relationship table vulnerability_tags. The execution output confirms that each table and index was successfully created without errors.

Figure 2 — Database schema successfully created using the SQL schema script



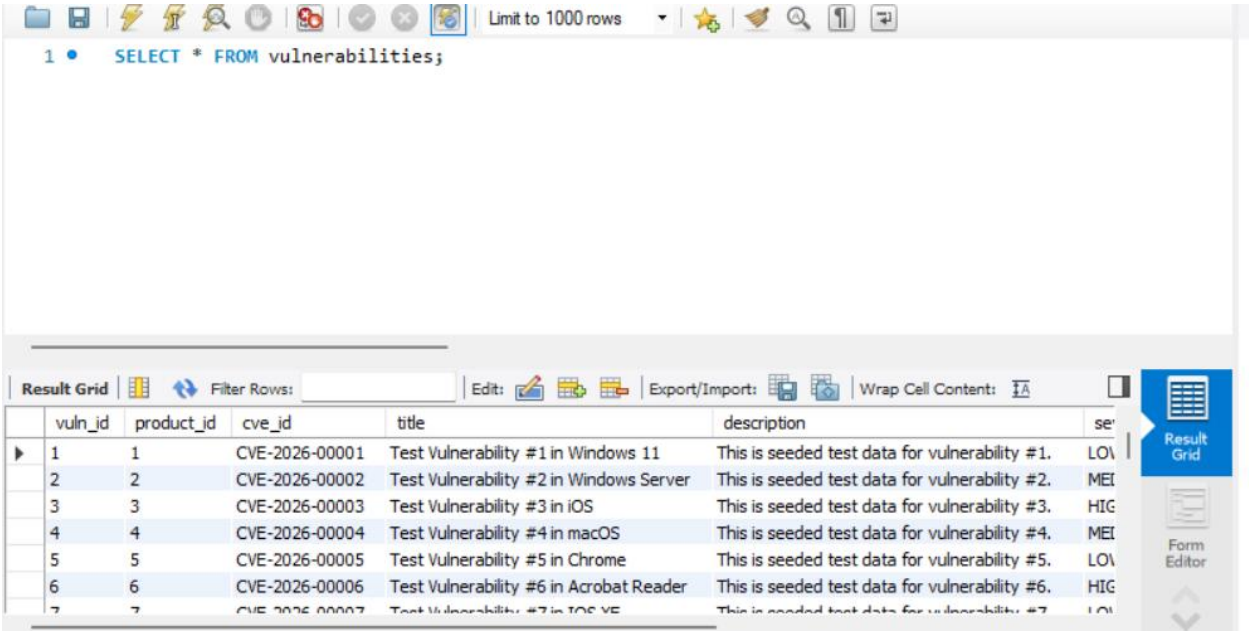
Tables created include:

- vendors
- products
- vulnerabilities
- vulnerability references
- tags
- vulnerability_tags

3.3 Insert Seed Data

After executing the 02_seed.sql script, sample data was inserted into the database tables. The query SELECT * FROM vulnerabilities; was executed in MySQL Workbench to verify that the records were successfully added. The results show multiple vulnerability records containing fields such as vuln_id, product_id, cve_id, title, description, and severity. This confirms that the database is populated with test data and ready for use by the application.

Figure 3 — Query results showing seeded vulnerability records in the database



vuln_id	product_id	cve_id	title	description	se
1	1	CVE-2026-00001	Test Vulnerability #1 in Windows 11	This is seeded test data for vulnerability #1.	LO
2	2	CVE-2026-00002	Test Vulnerability #2 in Windows Server	This is seeded test data for vulnerability #2.	MEI
3	3	CVE-2026-00003	Test Vulnerability #3 in iOS	This is seeded test data for vulnerability #3.	HIG
4	4	CVE-2026-00004	Test Vulnerability #4 in macOS	This is seeded test data for vulnerability #4.	MEI
5	5	CVE-2026-00005	Test Vulnerability #5 in Chrome	This is seeded test data for vulnerability #5.	LO
6	6	CVE-2026-00006	Test Vulnerability #6 in Acrobat Reader	This is seeded test data for vulnerability #6.	HIG

This confirms that the database is populated with initial data and ready to be used by the application.

3.4 Running the Service Layer

The service layer of the application was started using the command

python -m services.app.

The Flask development server successfully launched and began listening for HTTP requests on http://127.0.0.1:5000. This confirms that the service layer is operational and capable of processing requests from the client layer while communicating with the business and data layers of the system.

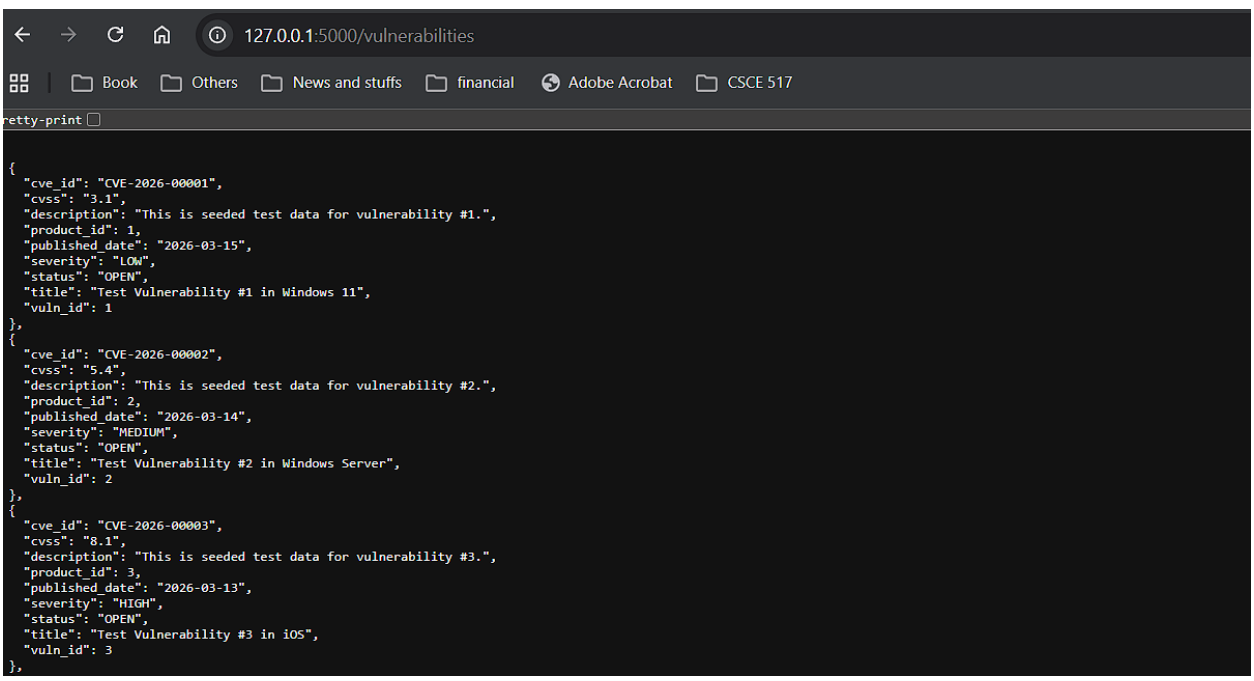
Figure 4 — Flask REST API service running locally

```
PS C:\CSCE548_Project> python -m services.app
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment
. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 111-677-240
```

3.5 Testing the API Endpoint

The REST API endpoint /vulnerabilities was accessed through a web browser while the Flask service was running. The API successfully returned vulnerability records stored in the MySQL database in JSON format. Each record includes fields such as vuln_id, cve_id, title, description, severity, status, and published_date. This demonstrates that the service layer is correctly retrieving data from the database through the business and data access layers.

Figure 5 — REST API endpoint returning vulnerability records in JSON format



The API successfully returned vulnerability records stored in the database, confirming that the service layer correctly communicates with the business and data layers.

---

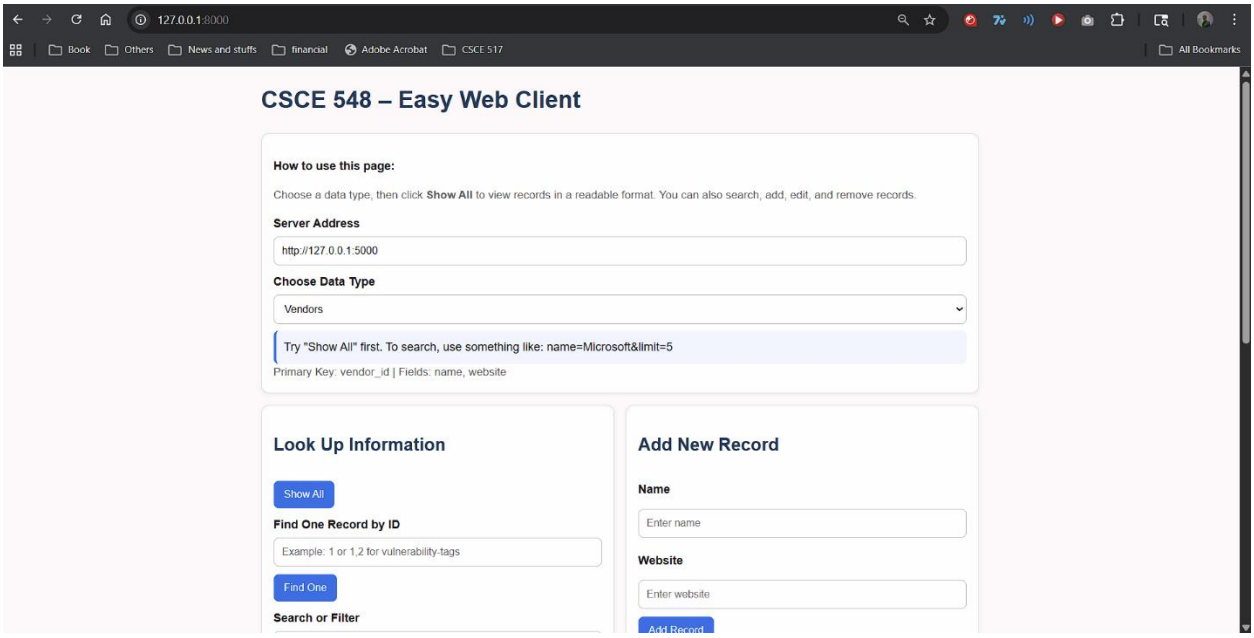
4. Client Interface

The project includes a web-based client that allows users to interact with the database in a user-friendly way.

Users can:

- View records
- Search records
- Add new records
- Update existing records
- Remove records

Figure 6 — Web client interface used to interact with the vulnerability database



The client communicates with the REST API using HTTP requests.

---

5. Full System Test

This section demonstrates the successful testing of all major application functions across the entire system.

5.1 Retrieve All Records

The client was used to retrieve all vendor records using the "Show All" feature.

Figure 7 — Client retrieving all vendor records from the API

Result

Request: GET http://127.0.0.1:5000/vendors

Request completed successfully.

Show / Hide Raw JSON

Vendor ID	Vendor Name	Website
5	Cisco	https://www.cisco.com
4	Adobe	https://www.adobe.com
3	Google	https://www.google.com
2	Apple	https://www.apple.com
1	Microsoft	https://www.microsoft.com

The client sends a GET request to the /vendors endpoint, and the API returns all records stored in the database.

5.2 Retrieve Record by ID

The client was used to retrieve specific records using the "Find One Record by ID" feature.

Figure 8 — Client retrieving individual vendor records by ID

Result

Request: GET http://127.0.0.1:5000/vendors/1

Request completed successfully.

Show / Hide Raw JSON

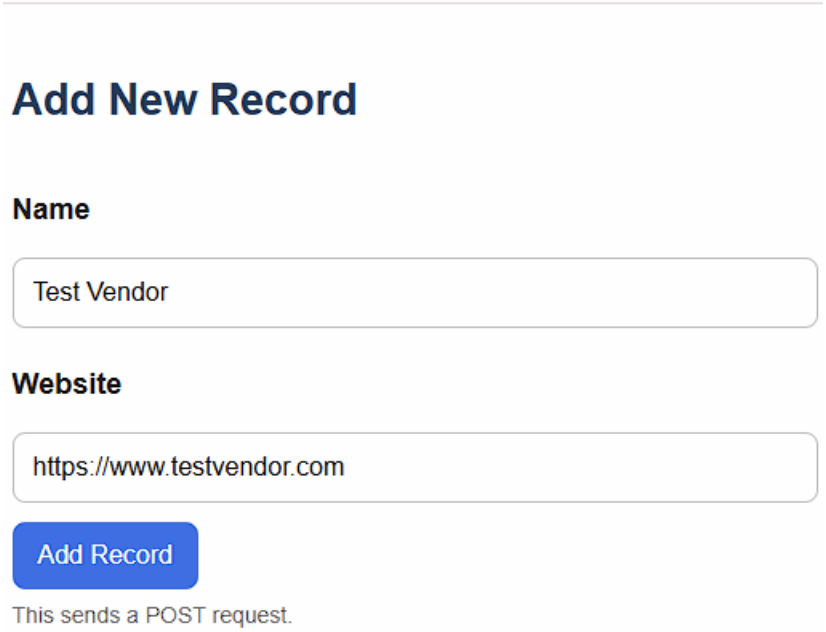
Vendor ID	Vendor Name	Website
1	Microsoft	https://www.microsoft.com

This demonstrates that the system supports retrieving specific records using their primary key.

5.3 Insert a New Record

A new vendor record was added using the client interface.

Figure 9 — Client inserting a new vendor record using a POST request



The client sends a POST request to the /vendors endpoint, and the record is successfully added to the database.

Figure 10 — New vendor was added to the table

Result

Request: GET http://127.0.0.1:5000/vendors

Request completed successfully.

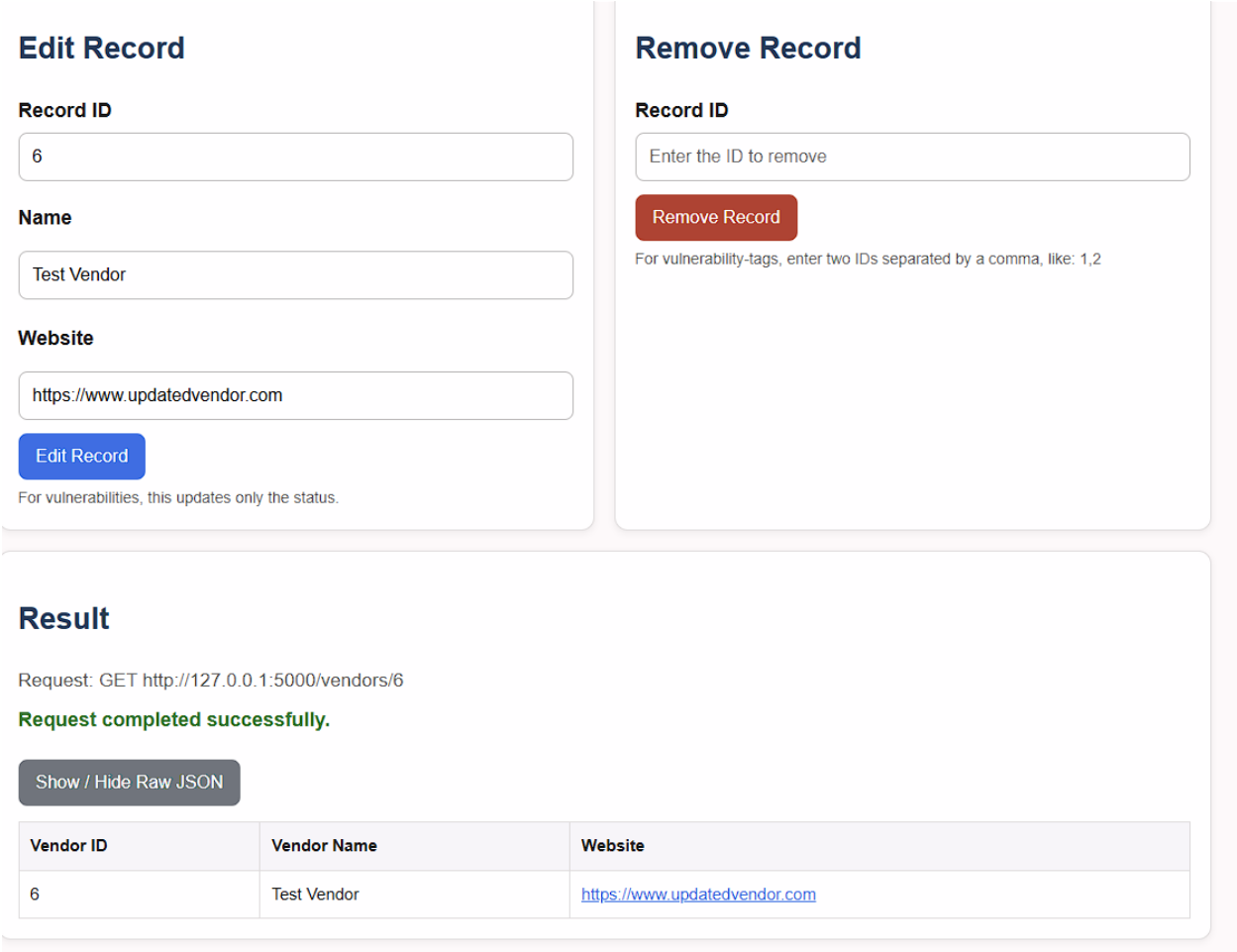
Show / Hide Raw JSON

Vendor ID	Vendor Name	Website
6	Test Vendor	https://www.testvendor.com
5	Cisco	https://www.cisco.com
4	Adobe	https://www.adobe.com
3	Google	https://www.google.com
2	Apple	https://www.apple.com
1	Microsoft	https://www.microsoft.com

5.4 Update an Existing Record

An existing vendor record was modified using the client interface.

Figure 11 — Client updating an existing vendor record

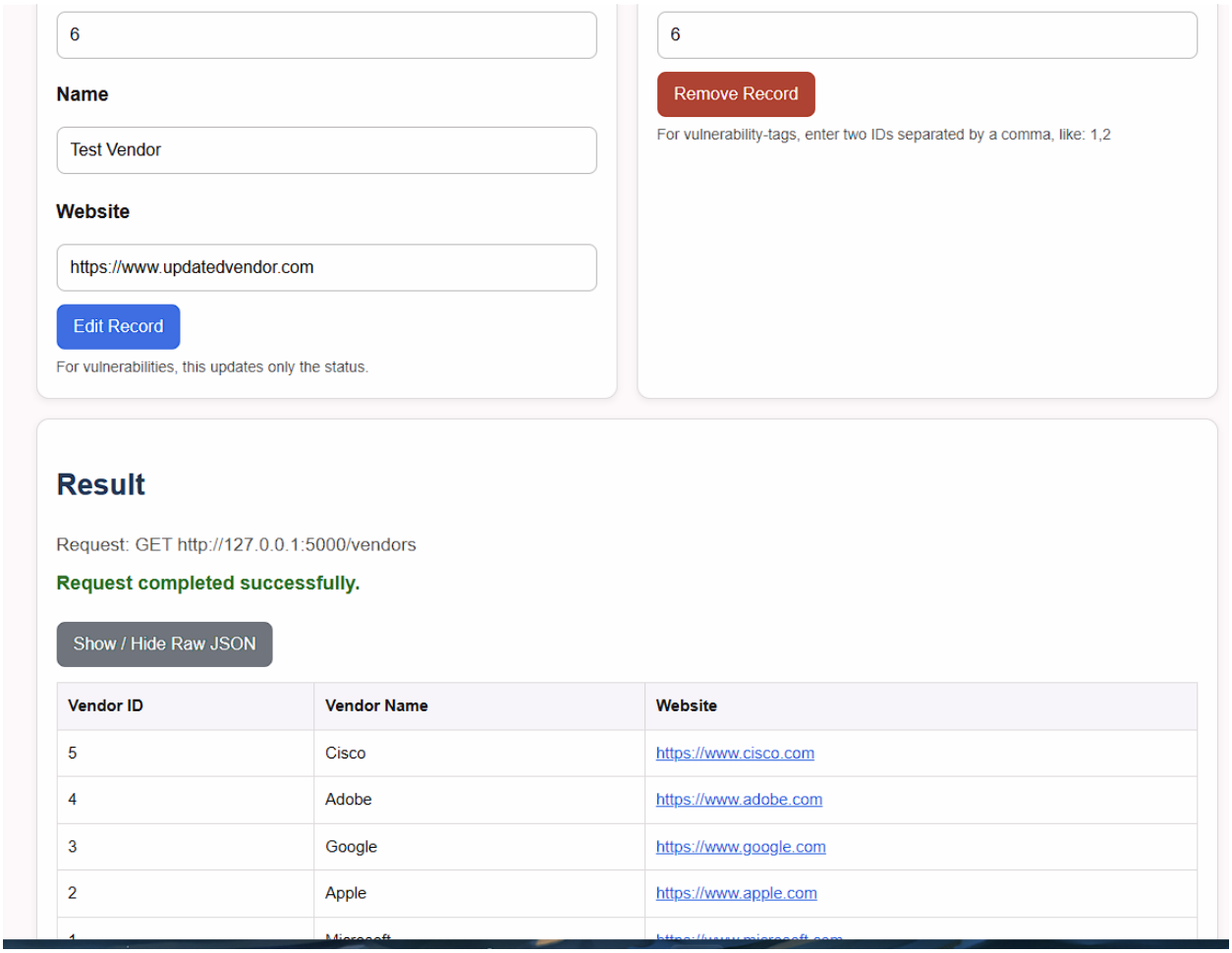


The update request was processed successfully, and the modified values were reflected in the database.

5.5 Delete a Record

A vendor record was removed using the client interface.

Figure 12 — Client removing a vendor record from the database



After the deletion request was processed, the record no longer appeared in the results, confirming the successful removal of the entry.

---

6. Conclusion

This project successfully demonstrates the development and deployment of a full-stack application using an n-tier architecture. The system integrates a web client, REST API, business logic layer, data access layer, and MySQL database.

All required functionality was tested and verified, including retrieving records, retrieving records by ID, inserting new records, updating existing records, and deleting records. The results confirm that the system operates correctly and that all layers communicate effectively.

The project demonstrates the practical implementation of modern web application architecture and the use of generative AI tools to assist in the development process.